
SISTEMA DE GESTIÓN DE CIUDADES Y BÚSQUEDA DE CAMINOS PARA MISIONES DE RESCATE Y EXTRACCIÓN

202501915 – Angel Emilio Ruíz López

Resumen

El presente trabajo se basa en el desarrollo de un sistema de control para la gestión de ciudades representadas mediante una malla ortogonal de celdas, así como la ejecución de misiones de rescate de unidades civiles y extracción de recursos. La solución y programación fue desarrollada en C# utilizando el paradigma de programación orientada a objetos y estructuras de datos implementadas propias, de acuerdo con las restricciones establecidas para el proyecto. La información de las ciudades y robots se obtiene mediante archivos XML, permitiendo construir cada ciudad y almacenar sus elementos para luego ejecutar las misiones. Para la búsqueda de caminos se implementó el algoritmo de búsqueda en profundidad (DFS), considerando las restricciones propias de cada tipo de robot y las condiciones de tránsito de las celdas. En el caso de los robots de combate, la capacidad disponible se modifica al atravesar unidades militares que pueden ser derrotadas. Finalmente, la herramienta Graphviz se utiliza para generar representaciones gráficas de las ciudades y de los caminos encontrados. La solución integra modelado, estructuras de datos, algoritmos y generación de reportes reunidos en un mismo sistema (program.cs)

Palabras clave

C#,
TDA
XML
DFS
Graphviz

Abstract

This work describes the development of a control system for managing of cities represented by an orthogonal grid of cells, as well as executing missions to rescue civilians and extract resources. The solution was programmed in C# using Object Oriented Programming paradigm and custom implemented data structures, in accordance with the project's restrictions. Cities and robot data are loaded from XML files, enabling constructions of each cities and the storages of its elements prior to mission execution. A (DFS) algorithm was implemented for pathfinding taking a specific restriction of the robot type and cell traversal conditions. For combat robots, available capacity when encountering and potentially defeating military units. Finally, Graphviz is used to generate a graphical representation of cities and the pathfinding. The solutions integrate a model, data structures, algorithms and report generation into a single system (program.cs)

Keywords

C#

ADT

XML

DFS

Graphviz

Introducción

El proyecto se basa en el desarrollo de un sistema capaz de administrar ciudades representadas mediante una malla ortogonal de celdas y ejecutar misiones utilizando robots específicos en el tipo de misión elegida por el usuario. Las ciudades pueden contener puntos de entrada, caminos, unidades civiles, recursos, unidades militares y celdas intransitables, por lo que la búsqueda de un camino depende de la estructura de la ciudad como de las características del robot seleccionado

La solución requiere integrar diferentes paradigmas de programación, entre ellos programación orientada a objetos, estructuras de datos abstractas, manejo de archivos XML, algoritmos de búsqueda y generación de reportes mediante Graphviz. El propósito de este proyecto es modelar la información, almacenar los datos y encontrar caminos que permitan completar la misión elegida por el usuario y este mismo pueda visualizar en pantalla la ciudad como el camino si pasaron por el respectivo filtro

Desarrollo del tema

1. Modelado y representación de la ciudad

Para representar esta estructura se diseñó la clase Ciudad, encargada de almacenar las características de la ciudad, como su nombre, cantidad de filas y

columnas y los elementos que conforman la malla. La ciudad se organiza mediante las estructuras Filas y Columnas, mientras que la clase Celda representa cada posición en la malla

Cada Celda contiene información de su posición, tipo de celda, capacidad de combate cuando corresponde a una unidad militar y estados utilizados durante la búsqueda de caminos. Además, las celdas mantienen referencias hacia sus posiciones vecinas: arriba, abajo, izquierda y derecha. Estas referencias permiten construir una malla ortogonal y facilitan el recorrido de la ciudad durante la ejecución del algoritmo de búsqueda. El diseño permite que una celda sea elemento conectado con las demás posiciones de la ciudad. Así, la estructura utilizada para representar la ciudad también proporciona las relaciones para realizar el recorrido de búsqueda

2. Estructuras de datos y programación orientada a objetos

Para administrar las ciudades y robots se implementaron listas enlazadas mediante las clases ListaCiudad, NodoCiudad, ListaRobot y NodoRobot. Estas estructuras permiten almacenar los objetos creados a partir de la configuración y realizar operaciones como agregar, buscar y actualizar elementos

Para almacenar el recorrido por la búsqueda se implementó una pila mediante Pila, NodoPila y Paso. La clase Paso representa una posición recorrida y almacena la celda correspondiente junto con capacidad utilizada en ese paso. La pila permite agregar y retirar pasos siguiendo el principio de LIFO de pilas, necesario para realizar el retroceso durante el algoritmo DFS

También se aplicó herencia en la representación de los robots. La clase abstracta Robot contiene las características básicas, mientras que Fighter y Rescue representan los dos tipos de robots. Los robots ChapinRescue están destinados a misiones de rescate

y no pueden enfrentarse a unidades militares, mientras que los ChapinFighter pueden combatir unidades militares cuando su capacidad de combate es mayor

3. Carga de configuración mediante XML

La clase LectorXML se encarga de interpretar la información del archivo XML y utilizarla para crear los objetos correspondientes. A partir de la información se crea una ciudad y construye su estructura de filas, columnas y celdas

Los caracteres de cada fila determinan el tipo de celda. El proyecto hace el uso de * para representar celdas intransitables, espacios para caminos transitables, E para puntos de entrada, C para unidades civiles y R para recursos. Las unidades militares se agregan mediante información adicional del archivo XML que indica su posición y capacidad de combate.

La información obtenida por el lectorXML se almacena en las listas propias del sistema. De esta forma, una vez termine la carga, los datos del archivo pueden utilizarse para ejecutar las operaciones del sistema

4. Búsqueda de caminos mediante DFS

Para solucionar este problema se implementó la búsqueda en profundidad (DFS). El algoritmo utiliza una pila para mantener el camino por aquellas celdas exploradas y válidas. Inicialmente se apila la celda de entrada y se marca como visitada. Luego se consulta la celda ubicada en la cima (actual) y se verifica si corresponde al objetivo de la misión

Si el objetivo todavía no ha sido cumplido, se revisan las celdas vecinas siguiendo el orden establecido por la implementación. Una celda candidata debe cumplir las condiciones de tránsito, no haber sido visitada antes y satisfacer las restricciones correspondientes al robot elegido y utilizado

En las misiones de rescate, el robot de tipo Rescue debe encontrar un camino seguro evitando las unidades militares. El proyecto establece que estos robots no pueden enfrentarlas ni sobrepasarlas.

En las misiones de extracción, el robot Fighter puede atravesar una unidad militar cuando su capacidad de combate es mayor que la capacidad de la unidad militar. Al derrotarla, su capacidad se reduce en el valor correspondiente de la Celda Unidad Militar.

Esto requiere que la búsqueda no solamente determine si una celda es transitable, sino que también considere el estado actual del robot en caso de ser de tipo Fighter. Por ello, durante el retroceso se debe restaurar la capacidad utilizada por el camino que se retrocede

Si todas las alternativas desde una posición son inválidas, el algoritmo realiza un retroceso mediante la pila y continúa explorando una alternativa mejor. Cuando la pila queda sin elementos significa que no se encontró una ruta válida y la misión debe considerarse imposible con un mensaje en pantalla

5. Generación de reportes mediante Graphviz

Para realizar esta función se desarrolló la clase GeneradorGraph, encargada de construir la representación para Graphviz. La generación comienza de la información almacenada en la ciudad y transforma las celdas en nodos de un archivo .dot.

La utilización de posiciones específicas permite conservar la distribución de filas y columnas de la ciudad. De esta forma, la representación mantiene la estructura de la malla original lo cual permite que el motor de Graphviz organice correctamente las celdas. Para el reporte del camino se utiliza adicionalmente la información almacenada por la búsqueda la cual está en la pila. Una vez encontrado el recorrido, las celdas pertenecientes al camino pueden diferenciarse visualmente dentro del mapa generado con un color amarillo y líneas que demuestran el orden el cual siguen como cuál es el camino

Este algoritmo permite obtener dos resultados principales: una representación gráfica de la ciudad y una representación gráfica de la ciudad junto con el camino encontrado para completar la misión. El uso de Graphviz permite convertir la información estructurada del programa en una representación que facilita comprobar el resultado de la búsqueda y el usuario pueda interpretar el resultado

El proyecto fue desarrollado y administrado mediante un repositorio de GitHub, utilizado para almacenar el código fuente, archivos de configuración y demás recursos del sistema, además de permitir mantener un historial de las modificaciones realizadas durante el desarrollo. GitHub define los repositorios como espacios destinados a almacenar el código, archivos e historial de revisiones de un proyecto

Conclusiones

El proyecto plantea el desarrollo de un sistema capaz de administrar ciudades representadas mediante una malla de celdas y ejecutar misiones utilizando robots especializados. Las ciudades pueden contener puntos de entrada, caminos, unidades civiles, recursos, unidades militares y celdas intransitables, por lo que la selección de un camino depende tanto de la estructura de la ciudad como de las características del robot utilizado

La solución requiere integrar diferentes conceptos de programación, entre ellos programación orientada a objetos, estructuras de datos abstractas, manejo de archivos XML, algoritmos de búsqueda y generación de reportes mediante Graphviz. El propósito de este trabajo es explicar las principales decisiones utilizadas para modelar la información, almacenar los datos y encontrar caminos que permitan completar las misiones

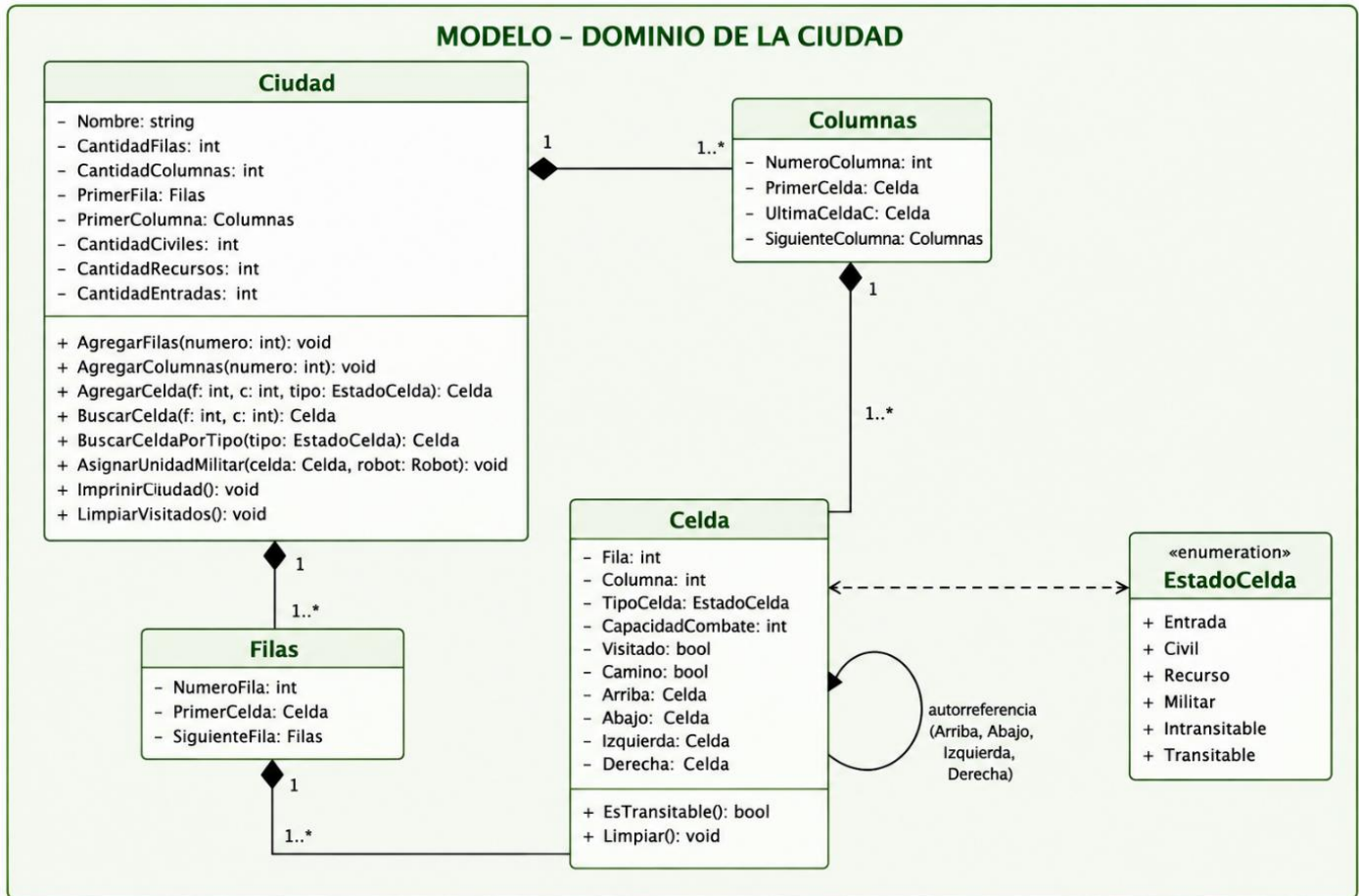
Referencias bibliográficas

- **Anthropic.** (2026). *Intro to Claude*. [Anthropic – Claude Documentation](#)
- **GitHub.** (s. f.). *Repositories documentation*. [GitHub Docs – Repositories](#)
- **Microsoft.** (2026). *C# documentation*. Microsoft Learn. [Microsoft Learn – C# Documentation](#)
- **World Wide Web Consortium (W3C).** (2008). *Extensible Markup Language (XML) 1.0 (Fifth Edition)*. [W3C – XML 1.0](#)
- **Graphviz.** (s. f.). *Documentation*. [Graphviz – Documentation](#)

Anexos:

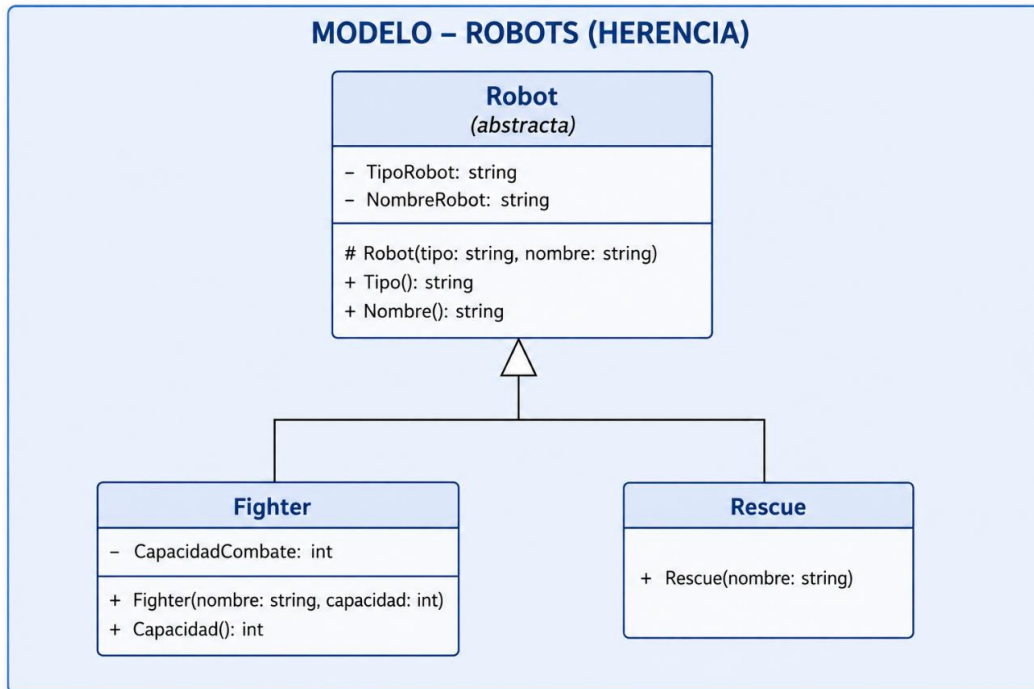
Diagramas de clase:

Figura 1. Clase para la creación del objeto ciudad



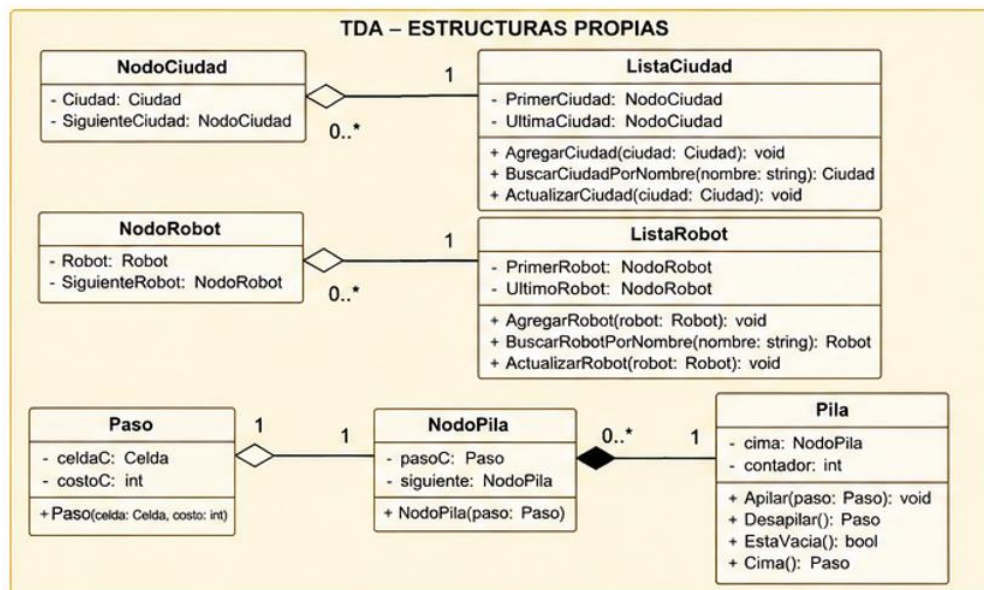
Fuente: Creación Propia

Figura 2. Clase para la creación del objeto robot



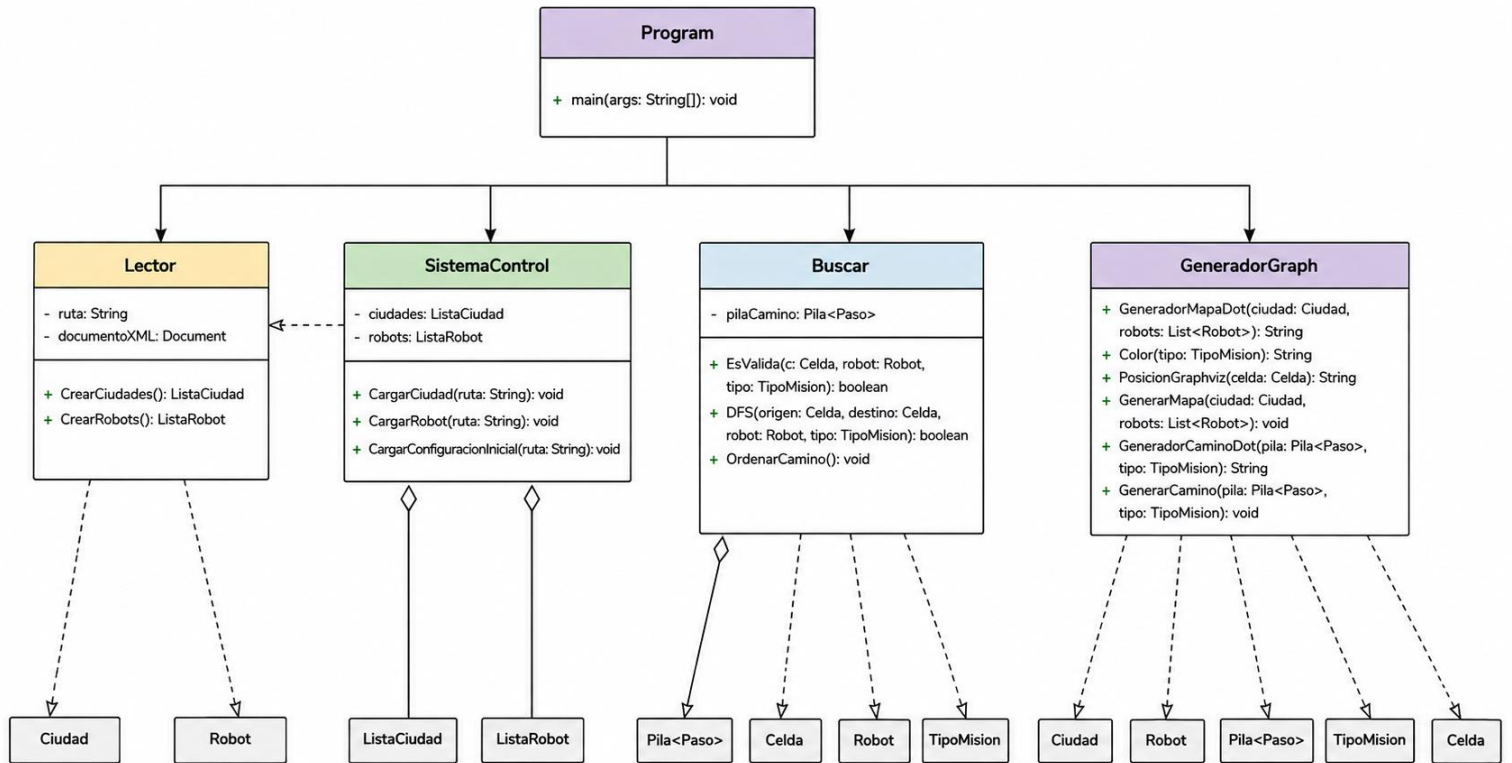
Fuente: Creación propia

Figura 3. Clase creación de estructura de datos (almacenamiento)



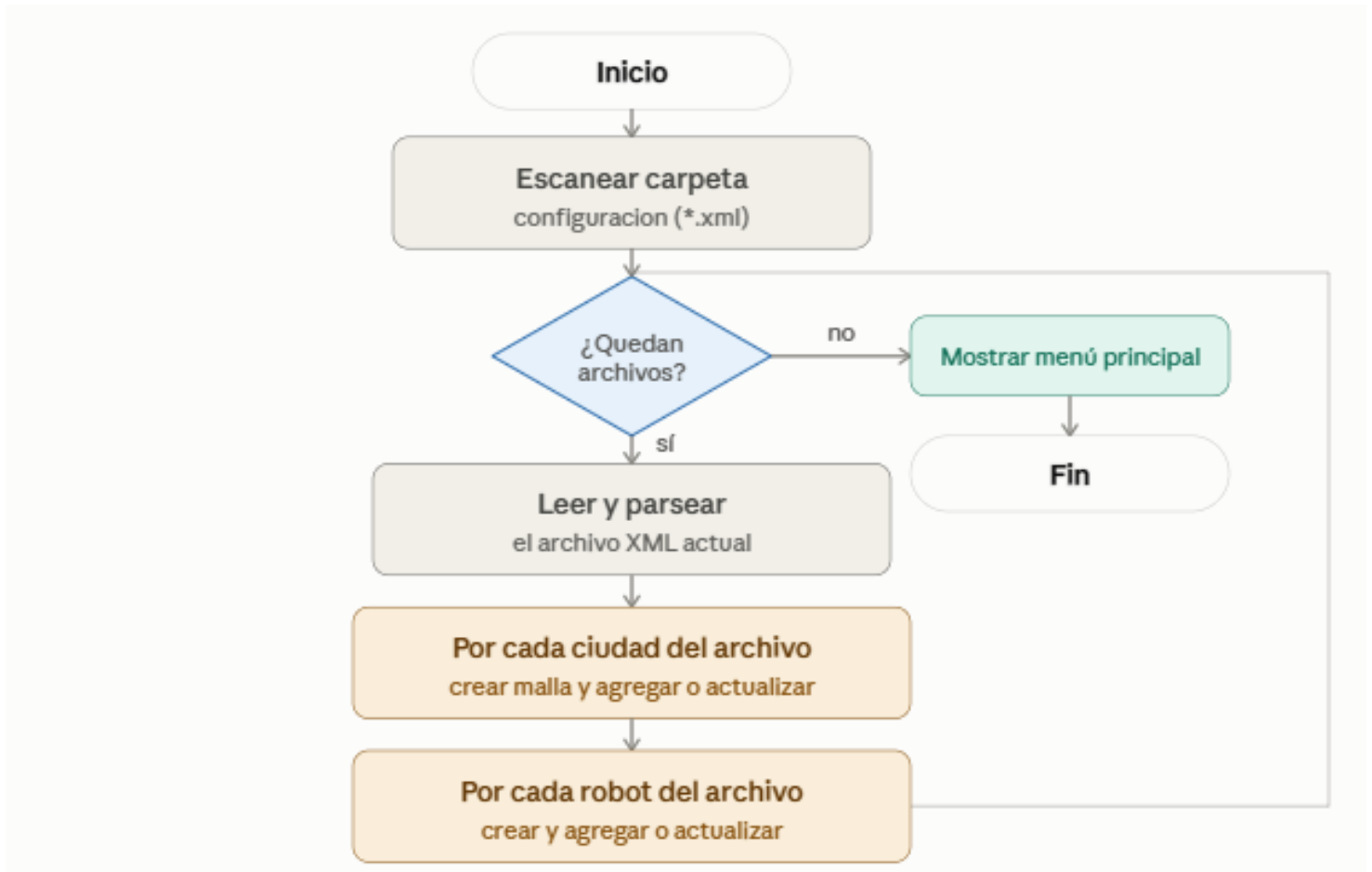
Fuente: Creación Propia

Figura 4. Program



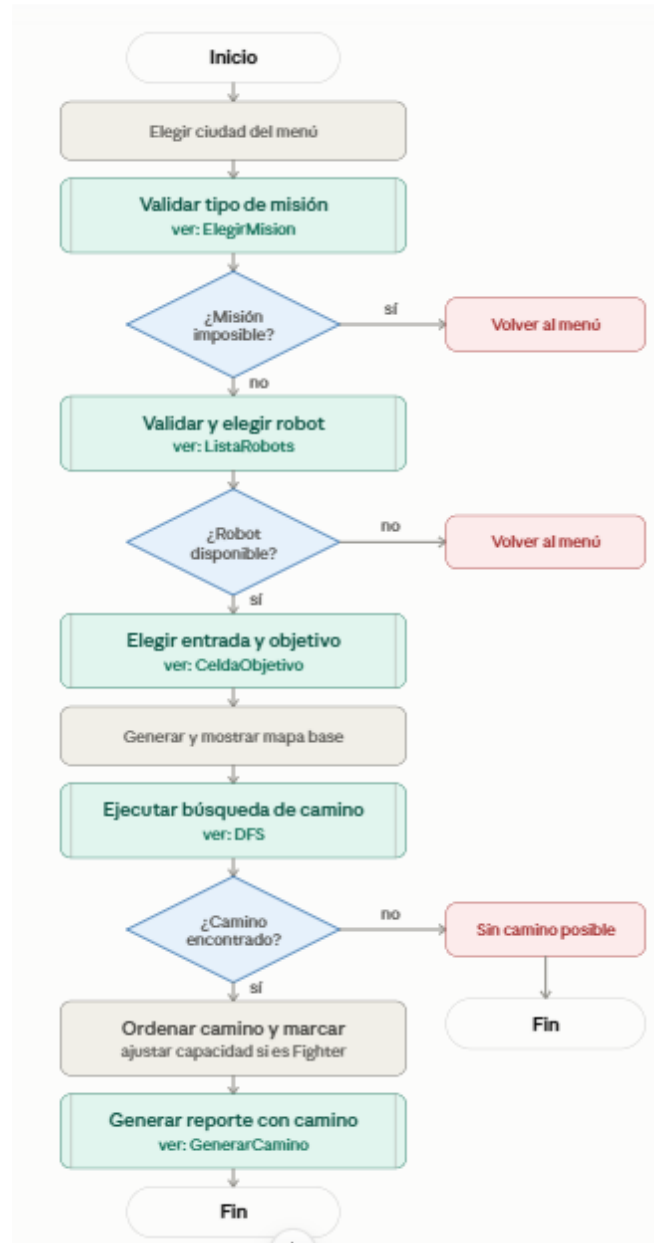
Fuentes: Creación propia

Figura 5. Flujo carga de datos



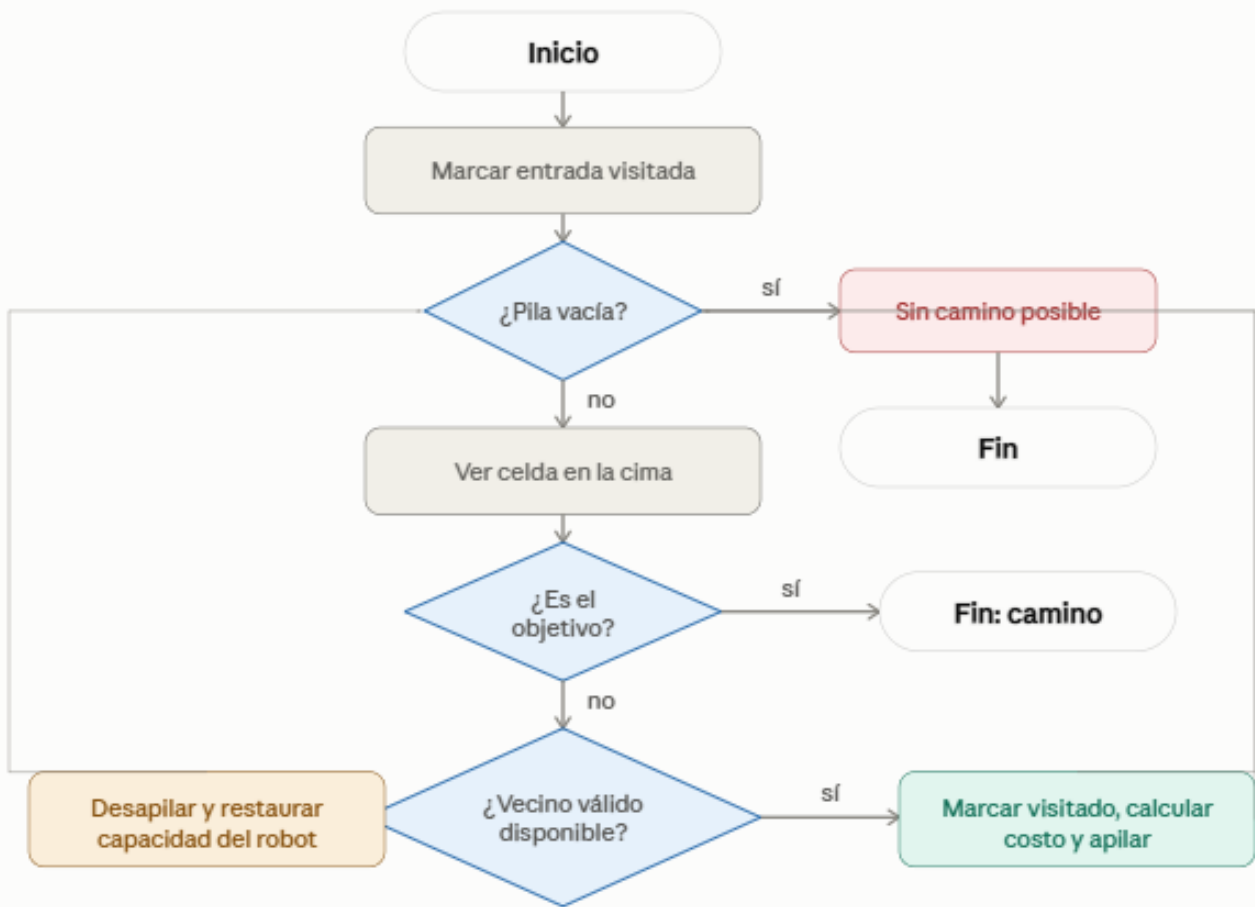
Fuentes: Creación propia

Figura 6.. Flujo general



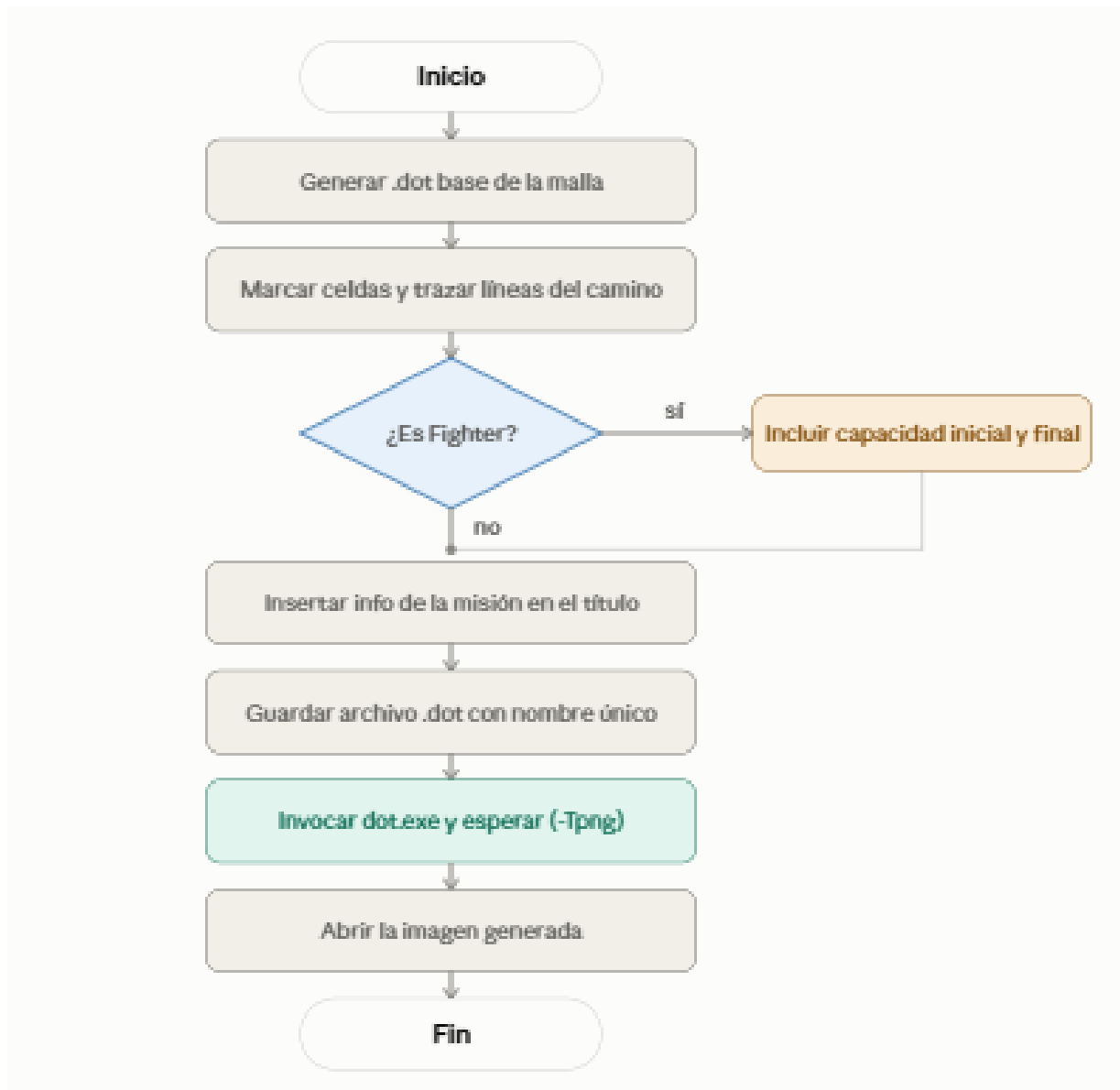
Fuentes: Creación propia

Figura 7. Flujo encontrar camino



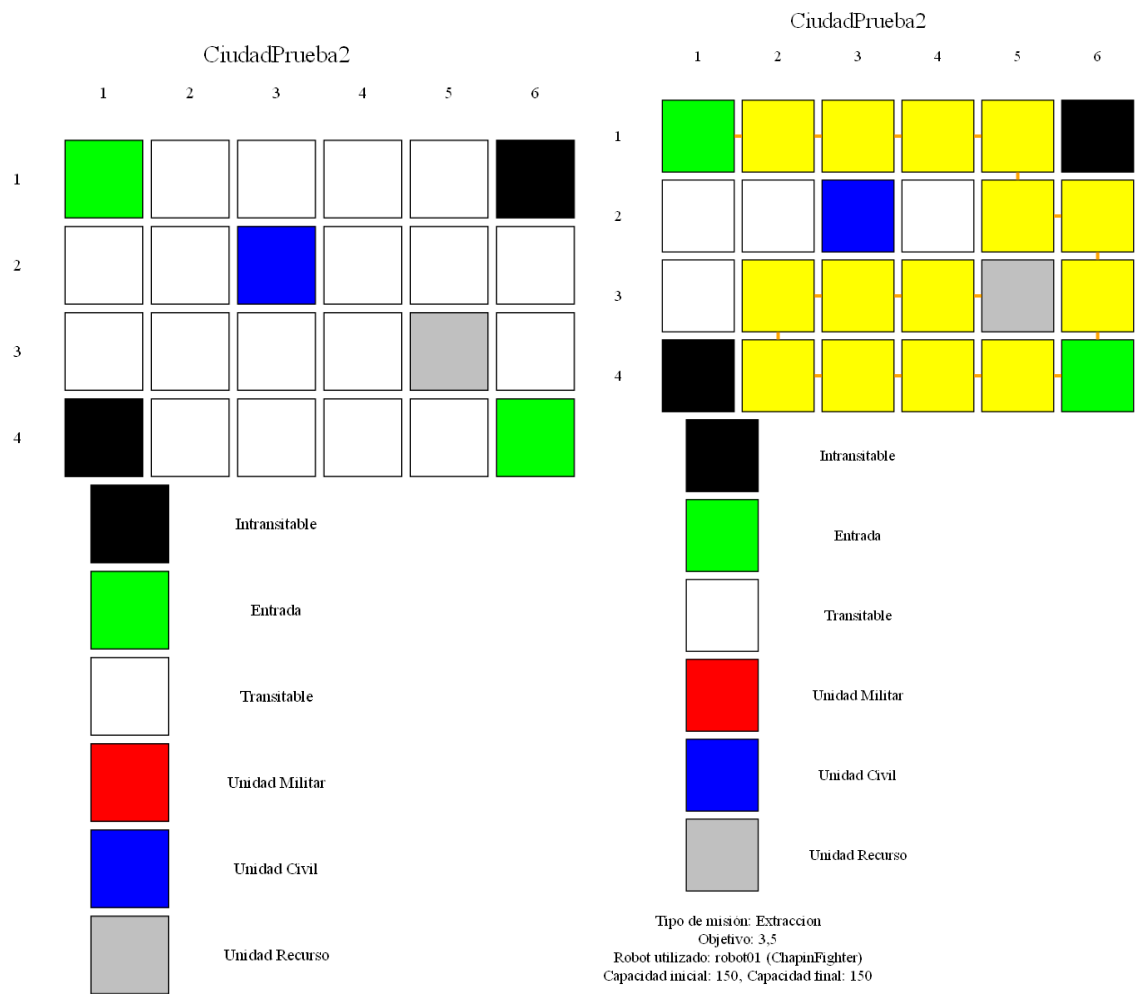
Fuentes: Creación propia

Figura 8. Flujo generar imagen por Graphviz



Fuentes: Creación propia

Figura 9. Resultado graficación



Fuentes: Herramienta Graphviz, resultado algoritmo en lenguaje c#